

# Chat portal internals

Embeddable React widget for text & voice chat. Runs in an iframe, receives a short-lived token from its host page, and streams to the server over one WebSocket.

React 19 + Vite

WebSocket pub/sub

AudioWorklet + VAD

MediaSource playback

## — WHAT IT IS

### A widget with two modalities

The widget is dropped onto a host page inside an iframe. It never touches the tenant API key — it gets a JWT from its parent window via `postMessage` and uses that to authenticate.

#### Text mode

- Type a message; server turn streams back token by token.
- Assistant replies render as Markdown (`react-markdown` + gfm).
- A stop button re-opens the last message for editing.

#### Voice mode

- Mic → voice-activity detection → PCM streamed up.
- Reply streamed back as MP3 and played via MediaSource.
- Speaking over the reply barges in and cancels it.

## — ANATOMY

# Provider tree

`ChatContext` consumes `WebSocketContext`, so provider order matters. Hooks hold the modality logic; components are thin.

```
<WebSocketProvider>      raw socket · subscribe(type → handler)
  <ChatProvider>          app state · token bootstrap · refs
    <App>
      <ChatWidget>        mode buttons · orchestration · expand/shrink
        └ <TextChatPanel>  message list + input · uses useTextChat
        └ <AudioStreamPlayer> MediaSource playback · uses useVoiceChat

hooks:  useTextChat · useVoiceChat · useAudioDevices
```

## COMMUNICATION

# Three channels

The widget talks over three distinct channels: `postMessage` to its host, REST to bootstrap, and a WebSocket for the conversation.

### HOST PAGE

#### **embed.js**

injects iframe · resize

#### **token responder**

give-token → set-token

`postMessage` · token & resize

### WIDGET (IFRAME)

#### **ChatContext**

token · config · refs

#### **WebSocketContext**

connect · subscribe

## CHAT SERVER

**/getConfig · /ws**

bootstrap + turn engine

**postMessage — host ↔ widget****▶ Widget → Host****give-token** on mount / expiry

Ask the parent for a fresh JWT.

**expand** chat opened

Ask the parent to grow the iframe.

**shrink** chat closed

Ask the parent to shrink the iframe.

**◀ Host → Widget****set-token** data: { token }

First time → run /getConfig. After → refresh token + re-auth socket.

**WebSocket — pub/sub bus**

WebSocketContext parses each incoming frame and fans it out to every handler registered for that `type`. Three consumers subscribe independently:

| TYPE        | CONSUMER                                                   | ACTION                                                                                              |
|-------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| control     | <code>useTextChat</code><br><code>AudioStreamPlayer</code> | Track the active <code>message_id</code> ; reset the audio buffer on <code>server_speaking</code> . |
| transcript  | <code>useTextChat</code>                                   | Append the STT text as a new user message.                                                          |
| data (text) | <code>useTextChat</code>                                   | Append the delta to the assistant bubble matching <code>message_id</code> .                         |

data (audio) **AudioStreamPlayer**

Base64-decode → queue → append to the MP3 SourceBuffer.

auth **ChatContext**

`session_expired` → re-request token;  
`auth_failed` → error.

## WebSocket — frames the widget sends

### ► Widget → Server

**auth** value: <jwt>

Sent automatically on socket open + on refresh.

**flag** {voice|text: "1"|"0"}

Enter / leave a modality.

**control** client\_speaking · client\_paused  
· client\_adds

Turn boundaries + barge-in/edit.

**text** value: <string>

Typed message.

**audio** JSON [ ...pcm bytes ]

A chunk of 24-bit PCM (voice).

### ◀ subscribed on the bus

**control** server\_processing · speaking ·  
paused

Turn lifecycle signals.

**transcript** text

What STT heard.

**data** text / audio

Streamed reply.

## AUTH BOOTSTRAP

## The token dance

The widget can't mint tokens; its host does. Everything downstream waits on this exchange — `App` shows a loader until both `token` and `conversationId` exist.

On mount the widget posts `give-token` to `window.parent` .

02

## Mint

The host's backend exchanges the tenant API key for a JWT  
( `/getChatToken` → `/generateToken` ) and posts `set-token` back.

03

## Bootstrap REST

The widget calls `GET /getConfig` with the Bearer token → learns its `identifier` ,  
`chat_type` , opener, and `background_color` (applied to `--primary` ).

04

## Connect WebSocket

Opens `/ws/{identifier}-{clientId}` and sends the `auth` frame on open.

05

## Refresh

On `session_expired` it re-posts `give-token` and re-auths the *same* socket — only the token rotates.

TOKEN TTL

**10 min**

server-enforced

CONVERSATIONID

**identifier-clientId**

= WS path segment

CLIENTID

**localStorage**

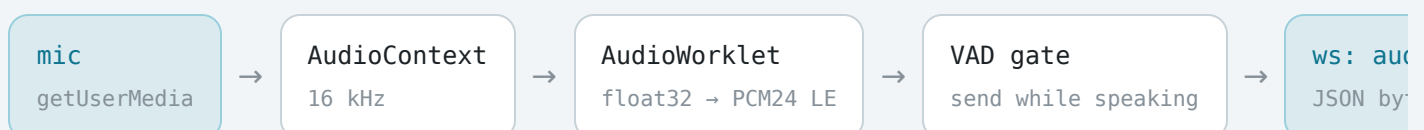
one session / browser

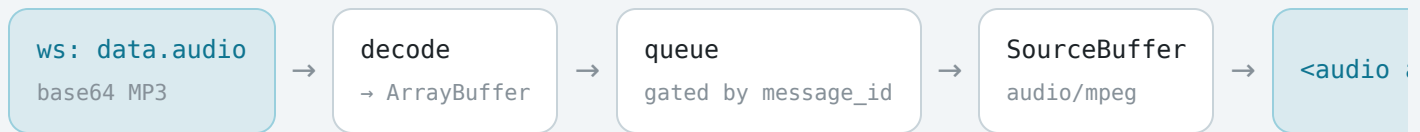
### AUDIO PIPELINE

## Uplink and downlink

Capture and playback are separate pipelines. Uplink is raw PCM gated by VAD; downlink is streamed MP3 assembled with Media Source Extensions.

UPLINK · CAPTURE





- ◆ **VAD (Silero v5).** `onSpeechStart` pauses playback and sends `client_speaking` — or `client_adds` if the server hasn't finished (barge-in). `onSpeechEnd` sends `client_paused`. The worklet only forwards audio while speech is detected, so silence isn't streamed.

## TEXT FLOW

# Sending & assembling a reply

01

## Optimistic render

Append the user message and an empty assistant bubble; store the bubble's id in `messageIdRef`.

02

## Send 3 frames

`client_speaking` (or `client_adds` when editing) → `text` → `client_paused`; set `serverNotYetAnsweredRef` to lock input.

03

## Stream in

`server_processing` sets the turn's `message_id`; each `data{text}` delta appends to the matching bubble. Stale ids are ignored.

04

## Stop / edit

The stop button reloads the last message into the input; the next send goes as `client_adds`, cancelling the server's in-flight turn.

# Start, run, stop

01

## Start

Connect → `flag {voice:"1"}` → mic permission → `AudioContext` + worklet → init & start VAD → `setRecording(true)` .

02

## Run

VAD drives control signals; the worklet streams PCM up while speaking; replies stream back through `AudioStreamPlayer` .

03

## Stop

`flag {voice:"0"}` → disconnect nodes, stop tracks, pause VAD, pause playback.

- ◆ Voice controls only appear when `/getConfig` reports `chat_type === "VOICE"` ; the server also rejects audio frames outside voice mode.

### STATE & REFS

## What lives where

Render-driving state sits in a reducer; fast-changing, render-irrelevant values live in refs shared across callbacks and WS handlers.

| REF                                  | CARRIES                                                           |
|--------------------------------------|-------------------------------------------------------------------|
| <code>audioRef</code>                | The hidden <code>&lt;audio&gt;</code> element (MediaSource sink). |
| <code>speechIdRef</code>             | <code>message_id</code> of the audio turn currently playing.      |
| <code>messageIdRef</code>            | <code>message_id</code> of the text turn being assembled.         |
| <code>serverNotYetAnsweredRef</code> | True between "sent" and "done" — gates input & barge-in.          |

flushMessageRef

Set on `server_paused`.

## BUILD & EMBED

# Vite app + Express embed host

## Widget build

- Base path `/chatwidget` (overridable via `APP_BASEPATH`).
- Dev proxy `/chat/` → `server` with `ws:true`; `CHAT_PROXY_STRIP=1` targets a local server.
- VAD worklet, Silero ONNX & onnxruntime WASM copied into the build.

## Embed host (ClientServingFiles)

- Express serves `embed.js` + the built widget.
- `/getChatToken` → `/generateToken`: the only place the API key is used.
- `embed.js` injects the iframe; the host page implements the `give-token` → `set-token` responder.

`aiservice_chat_portal/src` Companion doc: Server (backend) explainer.